File: `src/components/ChartPanel.jsx`

Category: React Component (Chart Container + Controls)
Runtime: Browser (React)
Primary parent: `src/components/ShellyDashboard.jsx`
Primary child: `src/components/LineGraph.jsx`
Uses: `SERIES_CATALOG`, `getTimeUnit()`

Overview

`ChartPanel` renders **one chart "panel"**. A panel consists of:

- Panel-level controls (remove chart, add series dropdown)
- Per-series controls (color, curvature/tension, thickness, remove series)
- Y-axis locking checkbox
- Optional "Info" popover showing latest values for all selected series
- The actual time-series chart (via `LineGraph`)

This component is **UI-focused**: it does not store global chart state itself.
Instead it calls state-management functions provided by `useChartPanels`.

Imports

- React: `useEffect`, `useMemo`, `useState`
- `LineGraph` from `./LineGraph.jsx`
- `getTimeUnit` from `../utils/time`
- `SERIES_CATALOG` from `../config/seriesCatalog`

Internal Helper Functions (defined in this file)

`labelWithUnit(key)`

Signature

```
labelWithUnit(key: string) → string
```

Purpose
Returns a human-readable label for a metric key.

Behavior

- Looks up the metric in `SERIES_CATALOG`
- If found:
  - returns `"Label Unit"` (e.g., `Voltage (R) V`) when unit exists
  - otherwise returns just `"Label"`
- If not found:
  - returns `key` (fallback)

`formatValue(v, decimals)`

Signature

```
formatValue(v: unknown, decimals: number | undefined) → string
```

Purpose
Formats metric values for display (used in the info popover).

Behavior

- `null`, `undefined`, `NaN` → `"N/A"`
- number + decimals specified → fixed decimal string
- otherwise → `String(v)`

`getLatestSeriesValue(latestPoint, seriesKey)`

Signature

```
getLatestSeriesValue(latestPoint: ParsedDataPoint, seriesKey: string) →
number | null
```

Purpose
Gets the latest value for a selected metric using its `valueAccessor`.

Behavior

- Finds the metric definition in `SERIES_CATALOG`
- Calls `valueAccessor(latestPoint)`
- Returns null if missing definition or if accessor throws

Exported Component

`ChartPanel(props)`

Signature

```
ChartPanel(props: ChartPanelProps) → JSX.Element
```

Properties (`ChartPanelProps`)

```
chart: ChartPanelState
selectedInterval: number
latestMs: number
filteredData: ParsedDataPoint[]
latestPoint: ParsedDataPoint | null
closeToken: number
actions: ChartPanelsAPI
styles: ChartPanelStyles | undefined
```

Notes on key properties

- `chart`: the full state object for this panel (id, selected series, axis lock info, etc.)
- `filteredData`: the time-window-filtered history used for plotting
- `latestPoint`: latest sample (for info popover "current values")
- `closeToken`: integer incremented by the parent to signal "outside click occurred"
- `actions`: functions returned by `useChartPanels`
- `styles`: theme object from `uiStyles` (defensive defaults exist)

Local State (UI-only)

`tensionPopup`

```
{ seriesKey: string } | null
```

Controls which series has its "Curvature" popover open.

`thicknessPopup`

```
{ seriesKey: string } | null
```

Controls which series has its "Thickness" popover open.

`infoOpen`

```
boolean
```

Controls whether the Info popover is visible.

These are intentionally local: they are pure UI state and do not belong in global chart state.

Effects

Close all local popovers on outside click signal

```
useEffect(() => { close popovers }, [closeToken])
```

Dependency array explanation

- The parent increments `closeToken` when the user clicks outside panels.
- When `closeToken` changes, this effect runs and closes:
  - curvature popup
  - thickness popup
  - info popup

This provides a clean "click outside closes popups" behavior without needing document-level event listeners.

Memoized Values (`useMemo`)

`addableKeys`

```
useMemo(() => getAddableKeys(chart), [getAddableKeys, chart])
```

List of metrics that are not yet selected in this chart (for dropdown).

`safePending`

```
useMemo(() => getSafePending(chart), [getSafePending, chart])
```

Ensures the dropdown value is always valid.

`chartSeries`

```
useMemo(() => buildChartSeries(chart), [buildChartSeries, chart])
```

Creates the series objects that `LineGraph` expects (includes `valueAccessor`, style settings, labels).

Why memoization matters here

- These computations are small but happen frequently.
- Memoization reduces unnecessary rebuilds when inputs are unchanged.

UI Behavior Summary

Panel-level controls

- **Remove chart**: calls `removeChart(chart.id)`
- **Metric dropdown**:
  - uses `addableKeys`
  - uses `safePending` as selected value
  - updates via `setPendingAddKey(chart.id, newValue)`
- **Add button**: calls `addSeriesToChart(chart.id)`

Y-axis lock

- Checkbox calls:

  ```
  toggleLockYAxis(chart.id, chart.currentYMin, chart.currentYMax)
  ```

- Disabled when no series are selected.
- Lock state displayed in the info popover as well.

Info popover ("i")

- Disabled if no series selected.
- When opened:
  - shows latest values for each selected metric
  - shows current auto-range min/max
  - shows lock status

Per-series "chip" controls

Each selected series is shown as a chip with:

- label
- color picker → `setSeriesColor(chart.id, key, color)`
- curvature button → shows range slider → `setSeriesTension(chart.id, key, tension)`
- thickness button → shows range slider → `setSeriesThickness(chart.id, key, thickness)`
- remove series button → `removeSeriesFromChart(chart.id, key)`

Chart Rendering (`LineGraph`)

`ChartPanel` renders:

```
LineGraph(
  dataPoints = filteredData,
  series = chartSeries,
  timeUnit = getTimeUnit(selectedInterval),
  intervalMs = selectedInterval,
  nowMs = latestMs,
  yAxis = { locked, min, max },
  onAutoYRange = (min,max) => setChartAutoYRange(chart.id, min, max)
)
```

Key responsibilities delegated to `LineGraph`

- Data filtering inside interval window (secondary safety)
- Downsampling / point reduction
- Auto Y-range calculation
- Chart.js rendering

Feedback loop: `onAutoYRange`

`LineGraph` computes an auto Y-range and reports it upward.
`ChartPanel` forwards this to the global chart state manager via `setChartAutoYRange`.

This enables Y-axis locking to capture a stable range.

Side effects

- No external side effects (no network, no timers)
- React state updates (local UI state)
- Calls provided state-management actions (updates global chart state)